

NEOSTATS DATA ENGINEER ASSIGNMENT

AIRLINE USE CASE – SUBMISSION

NAME: RESHMI .A

REG.NO: 22MID0169

VIT - VELLORE CAMPUS

Maid id: reshmiashok1806@gmail.com

1. PROJECT OVERVIEW

This project is based on a case study done for ASG Airlines where I worked with raw airline data comprising flights, bookings, passengers and payments. The data was in one Excel workbook with 4 separate sheets and there were a number of quality issues that needed to be corrected before the data could be used for any type of reporting. So I wanted to take this data and clean it up, check it for errors, calculate some useful numbers like flight duration, and then build a Power BI dashboard on top of it so that someone looking at ASG Airlines' operations could actually understand what was going on – which routes are busiest, how the airlines compare, how much revenue is coming in, and so on. This document describes what I did at each step of the project, why I did it like that, and the final product.

2. PROBLEM STATEMENT

The raw data set provided by ASG Airlines was not clean. As I went through it I found a few issues, for example:

- Some flight records had airline value as “UNKNOWN” instead of the actual airline name.
- Some flights had a duration field that did not match the actual difference between departure & arrival times.
- One flight record had a departure time that was after the arrival time, which is impossible.
- There was one flight ID that appeared multiple times in the data. This is not supposed to be the case as flight IDs are supposed to be unique.
- Some bookings did not have a status while others had status “INVALID”.
- Some bookings had a flight id that did not exist in the cleaned flights table.
- Some records of passengers did not contain last names.
- Some payment amounts were not numeric – they contained text values instead of a correct amount.

There was also personal information in the bookings data, such as passport numbers and emergency contact details, that should not be visible directly in an analytics or reporting layer, which is why I couldn't simply hook up the raw data to Power BI and expect the numbers to be right. First I had to run a proper cleaning and validation step.

3. OBJECTIVES

- Load the raw data relating to ASG Airlines (including flights, bookings, passengers and payments) from the Excel workbook.
- Find and fix data problems like duplicates, missing values, and inconsistent airline names.
- Work out the flight duration based on the departure and arrival times and then compare it with the duration listed in the data.
- Make sure that flights which cross midnight (i.e. overnight flights) have their duration calculated correctly and not as a negative value.
 - Check the data by identifying and either flagging or deleting records that don't make sense (for instance, those with an arrival time before departure or those that refer to a flight which doesn't exist).
- Create separate "analytics-safe" versions of the passenger and booking data which exclude personal information, so that the dashboard layer has only the information that is actually necessary.
- Work out a number of simple business key performance indicators, such as the average length of a flight, the most popular routes, the cancellation rate, and the revenue.
- Create a Power BI dashboard enabling the cleaned data to be explored visually.

4. DATASET DESCRIPTION

The source data was an Excel workbook (UseCase - Airlines.xlsx) with four sheets. Each sheet represents one part of the airline's operations:

SHEET	DESCRIPTION	KEYCOLUMNS
Flights	One row per flight, with route and timing details.	flight_id, airline, source, destination, departure_time, arrival_time, duration
Bookings	One row per booking made by a passenger for a flight.	booking_id, passenger_id, flight_id, booking_date, status, passport_number, seat_number, emergency_contact_name, emergency_contact_phone

Passenger	One row per passenger.	passenger_id, first_name, last_name, age, gender
Payment	One row per payment made against a booking.	payment_id, booking_id, payment_method, amount

After cleaning the flights table ended up with 1,002 records (three records were removed as invalid – more on this in the Data Validation section). The bookings table, the passengers table and the payments table each had 1,000, to 1,039 records.

5. DATA INGESTION

I used Python and the pandas library to bring in the data. Because all four tables were in sheets inside an Excel file I opened the file once using `pd.ExcelFile()`. Then read each sheet one by one using `pd.read_excel()` making sure to set the `sheet_name` parameter for flights, for bookings for passengers and for payments.

Before I started cleaning anything I looked at each sheet by itself – I checked the number of rows and columns the names of the columns the types of data any values and any duplicate rows. This allowed me to see clearly what needed to be fixed before I began the cleaning process.

6. DATA CLEANING AND PREPROCESSING

○ *FLIGHTS*

- I removed duplicate rows using `drop_duplicates()`. The airline column had values, including "UNKNOWN" entries.
- I noticed that every `flight_id` starts with a 2-character airline prefix, such, as AI, SJ, UK or 6F. So I extracted this prefix. Mapped it to the correct airline name using a simple dictionary: AI → Air India, SJ → SpiceJet, UK → Vistara, 6F → IndiGo.
- This created an consistent airline column, instead of depending on the original values that were sometimes wrong.

○ *BOOKINGS*

- Some bookings had a missing status. I filled these with “UNKNOWN” instead of leaving them blank, so at least it is clear that the value was not recorded rather than accidentally dropped.

- I kept the “INVALID” status bookings in the data (rather than deleting them) since they still represent something that happened in the source system, but I could identify and count them separately for data quality reporting.
- *PASSENGERS*
 - Some passengers had a missing last name. I filled these with “UNKNOWN” so the field is never blank.
 - I checked for duplicate passenger IDs and for the range of the age column to get a sense of data quality (for example, checking the minimum and maximum age values, and confirming gender only had the expected values).
- *PAYMENTS*
 - The amount column had -numeric entries in some rows—some contained text instead of numbers. I used `pd.to_numeric()` with `errors="coerce"` to handle this. This approach converts anything that can't be turned into a number into a missing value (like NaN) of causing an error that stops everything.
 - I then made a new column called `amount_quality`. This column marks each row as `VALID`, `MISSING` or `INVALID` based on the amount value. It's `VALID` if the number converted cleanly `MISSING` if the original was blank or empty and `INVALID` if it was clearly wrong (like text that can't be turned into a number). This helps me keep track of data quality. When I calculate revenue later I can just filter for `VALID` rows. Avoid using bad or incomplete data.

7. DATA TRANSFORMATION

Once the raw values were cleaned I created a few columns that made later analysis easier:

- `flight_prefix` – the first two characters of `flight_id` used to work out the airline.
- `Airline (cleaned)` – replaced the `airline` column with the value derived from the flight prefix mapping.
- `Duration_minutes` – the validated flight duration converted into minutes so it is easier to use in charts and averages.
- `Flight_reference_valid` (in bookings) – a True/ flag showing whether a bookings `flight_id` actually exists in the cleaned flights table.
- `Amount_quality` (in payments) – a label showing whether a payment amount was usable for revenue calculations.

These extra columns don't change the meaning of the data they just make the cleaned data easier to work with in Power BI and, in any further Python analysis.

8. FLIGHT DURATION CALCULATION

Each flight record already had a duration field. I did not fully trust it. So I calculated the duration myself using the timestamps:

calculated_duration = arrival_time - departure_time.

I then compared this calculated duration with the duration value that was already in the data. I allowed a small tolerance of 1 second to account for rounding differences. If the two values matched within that tolerance and the calculated duration was zero or positive I considered the record valid. Then I used the duration converted to minutes as the final duration_minutes value.

Across the dataset the average flight duration turned out to be around 164.5 minutes. This number also shows up as one of the KPI cards, on the Power BI dashboard.

9. OVERNIGHT/CROSS-DAY FLIGHT HANDLING

Since departure_time and arrival_time in this dataset are full timestamps (they include both the date and the time), genuine overnight flights are not actually a special case – if a flight departs at 11:38 PM and lands at 2:32 AM, the arrival_time already carries the next day's date. So a simple subtraction (arrival_time - departure_time) gives the correct positive duration automatically, without me needing to add any extra “next day” logic.

The only place this became a real problem was with one specific flight (SJ192), where the arrival_time in the data was actually earlier than the departure_time by a full day – in other words, the arrival date looked like it was recorded one day before the departure date. This wasn't a genuine overnight flight, it was a data error. Because the calculated duration came out negative, this record failed the duration validation check and was moved to the invalid flights output instead of being included in the clean data.

10. DATA VALIDATION

After cleaning, I ran a set of validation checks on the flights table:

- Duration validation – checked that calculated_duration was not negative and matched the stated duration within a 1-second tolerance.

- Duplicate flight ID check – checked whether the same flight_id appeared more than once. I found that 6F250 appeared twice, so both occurrences were treated as unreliable and moved out of the clean data.

Combining both checks, three flight records ended up in the invalid_flights output: SJ192 (failed the duration/timestamp check) and two rows for 6F250 (duplicate flight ID). The final cleaned_flights table has 1,002 valid records.

I also validated the bookings table by checking whether each booking's flight_id actually exists in the cleaned flights table. This produced the flight_reference_valid flag. Out of the bookings, 997 had a valid flight reference and 3 did not – these 3 were the bookings that pointed to the flight IDs that got removed during flight validation. This is shown on the dashboard as the “Flight Reference Validation” chart.

11. DATA MODELLING

The cleaned data follows a simple relational structure with four tables, connected through IDs:

- flights (flight_id as the key) holds route and timing information.
- bookings connects a passenger to a flight, using passenger_id and flight_id as foreign keys, with booking_id as its own key.
- passengers (passenger_id as the key) holds passenger details.
- payments connects to a booking using booking_id as a foreign key, with payment_id as its own key.

This means bookings is really the central table – it links flights, passengers, and (indirectly, through it) payments together. This structure is described in more detail in the Data Model section below.

12. BUSINESS KPIS

Once the data was cleaned, I calculated a few KPIs that summarise ASG Airlines' operations. These are the same numbers that feed into the Power BI dashboard:

KPI	VALUE
Total flights (valid)	1.002
Total bookings	1,000
Total unique routes	30

Average flight duration	164.5 minutes
Cancellation rate (bookings with status = CANCELLED)	31.4%
Total valid payment revenue	₹7,385,142.98 (≈ 7.39M)
Average valid payment amount	₹8,009.92

Flights by airline came out fairly close to each other: IndiGo (271), Air India (255), SpiceJet (246), and Vistara (230). The busiest route by far was Mumbai to Kolkata (BOM → CCU) with 90 flights, followed by Kolkata to Delhi (CCU → DEL) with 72 flights, and Chennai to Bengaluru (MAA → BLR) with 65 flights.

Revenue by payment method, using only the VALID payments, worked out to roughly: UPI ₹2.62M, CARD ₹2.40M, and NETBANKING ₹2.37M.

13. POWER BI DASHBOARD

Dashboard 1:

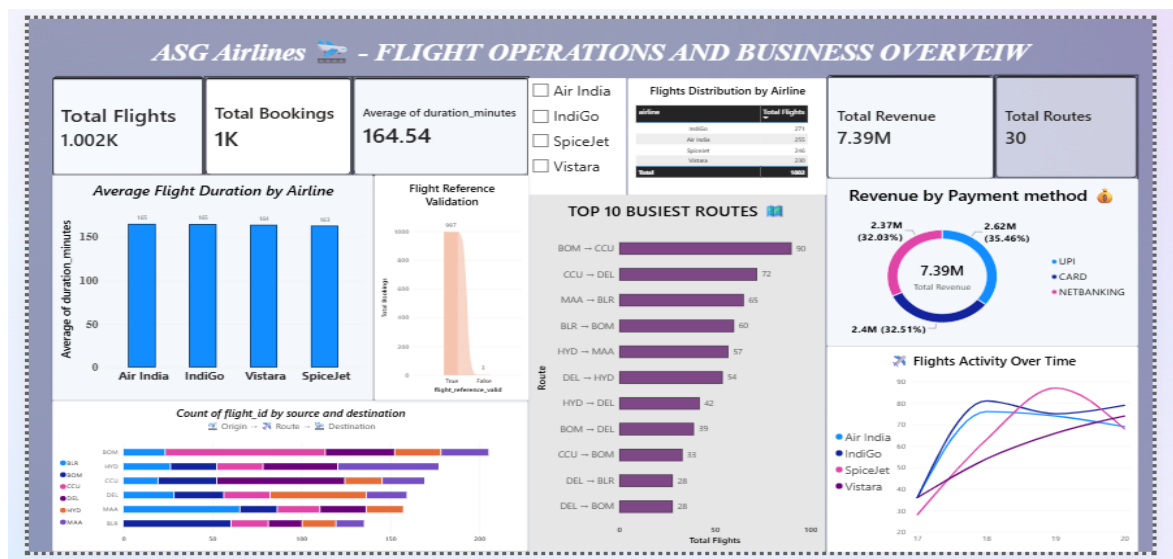


Figure 1 Flight Operations and Business overview Dashboard 1

Description:

This dashboard shows an overview of ASG Airlines flight operations and business performance. It lists KPIs such, as total flights, bookings, revenue, airlines, routes and average flight duration.

Interactive visuals help us understand airline distribution, the busiest routes, payment methods, flight activity and flight-reference validation.

Dashboard 2:

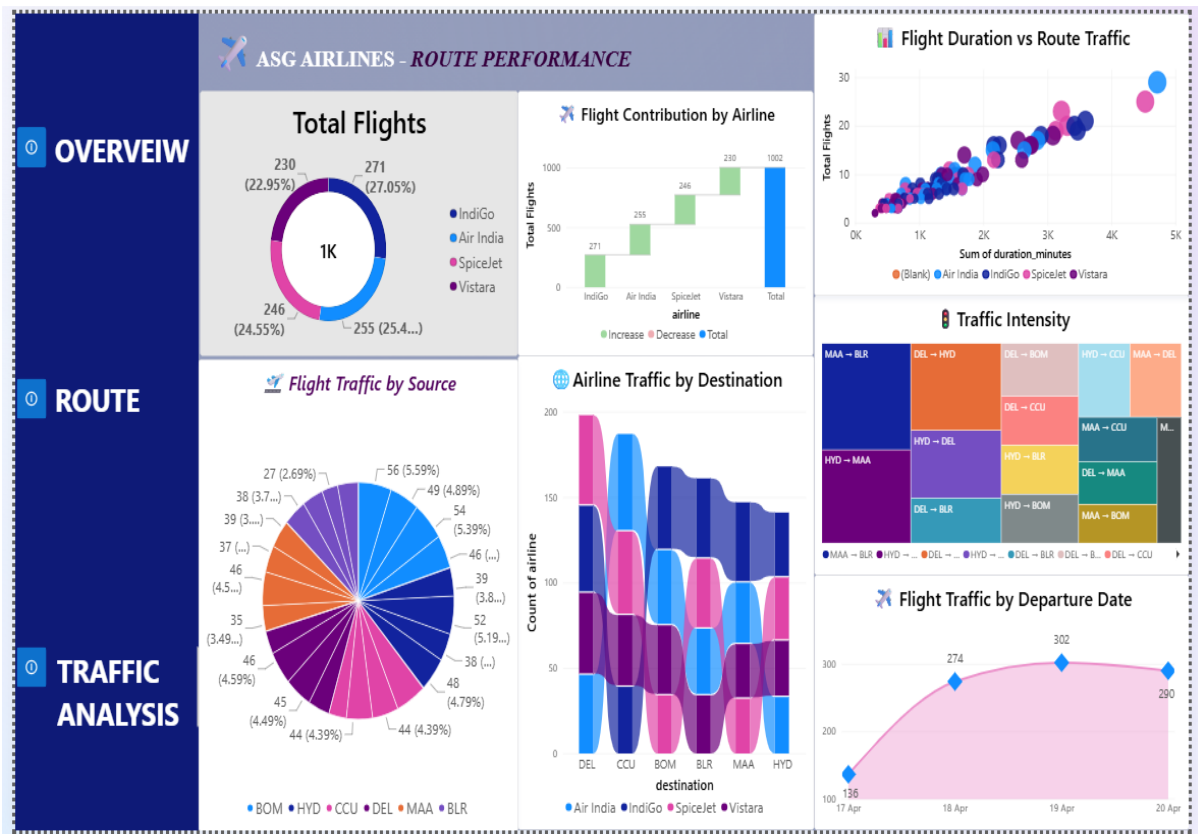


Figure 2 Route Performance Dashboard 2

Description:

This dashboard focuses on route-level performance. Dashboard shows route traffic, busiest flight routes, traffic intensity, airline-wise destination traffic and the link, between flight duration and route traffic. These insights help find high-traffic routes and help understand route-level patterns.

Dashboard 3:

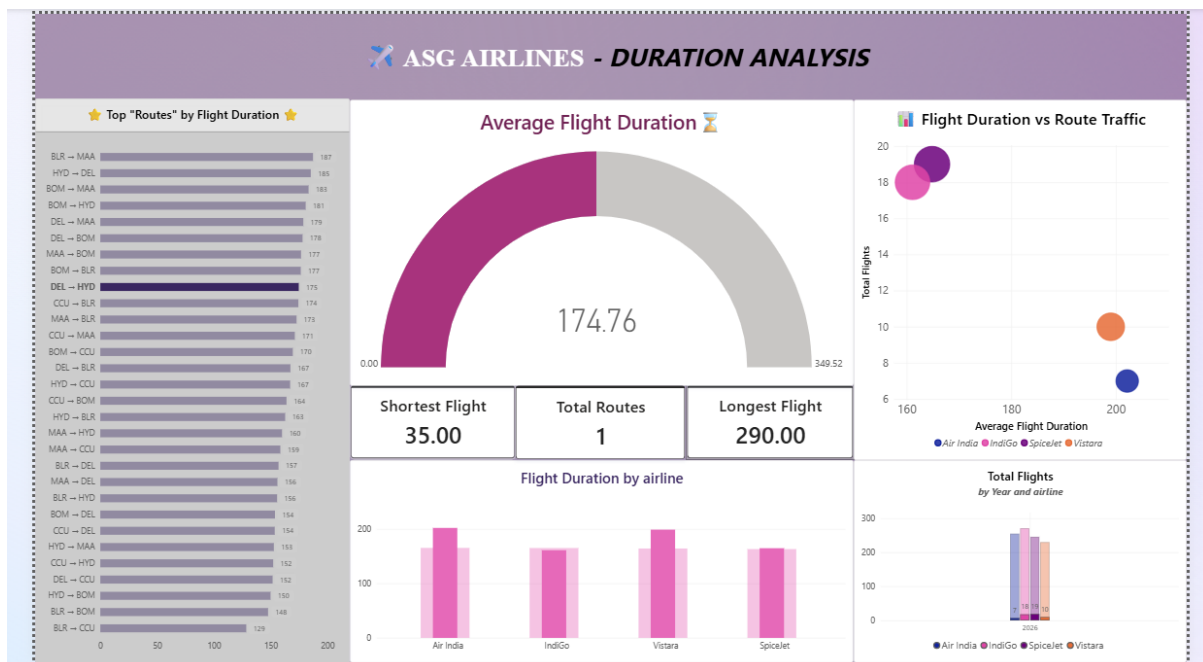


Figure 3 Duration Analysis Dashboard 3

Description:

This dashboard looks at flight duration patterns across airlines and routes. I find this dashboard useful because it shows the minimum and maximum flight durations, for airlines and routes. It also shows how flight duration patterns relate to traffic volume. The visuals help spot variations and unusual flight duration patterns.

Dashboard 4:

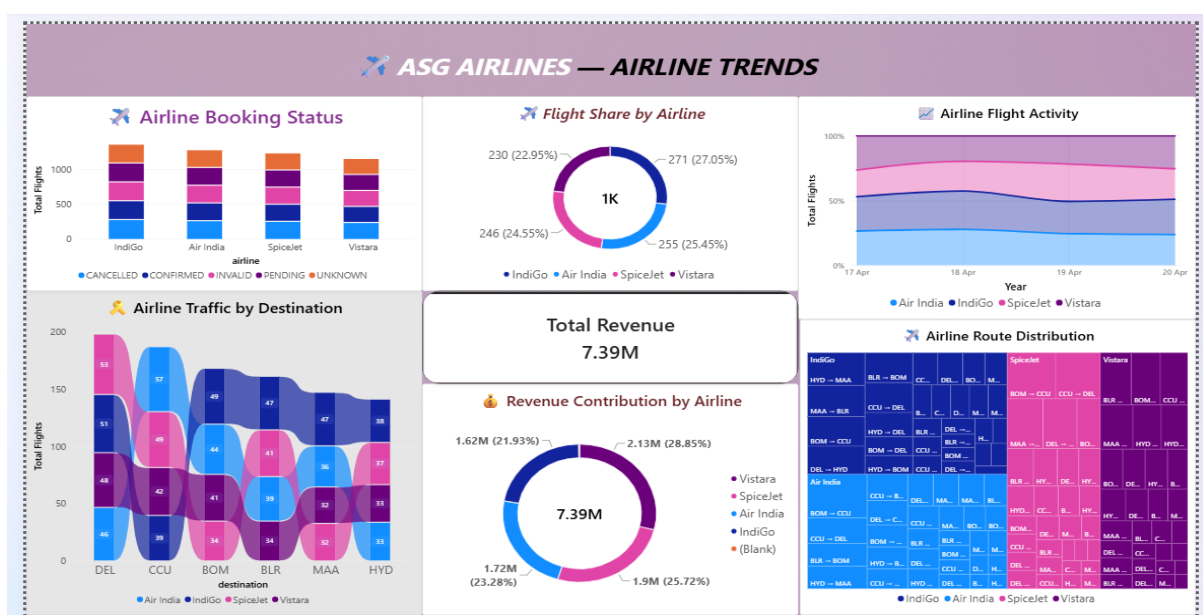


Figure 4 Airlines Trend Dashboard 4

Description:

This dashboard shows how different airlines perform in operations and business. It lists flight share booking status, destination traffic, route distribution, flight activity and revenue contribution. These visualizations help compare airline performance and spot the traffic contributors.

Dashboard 5:

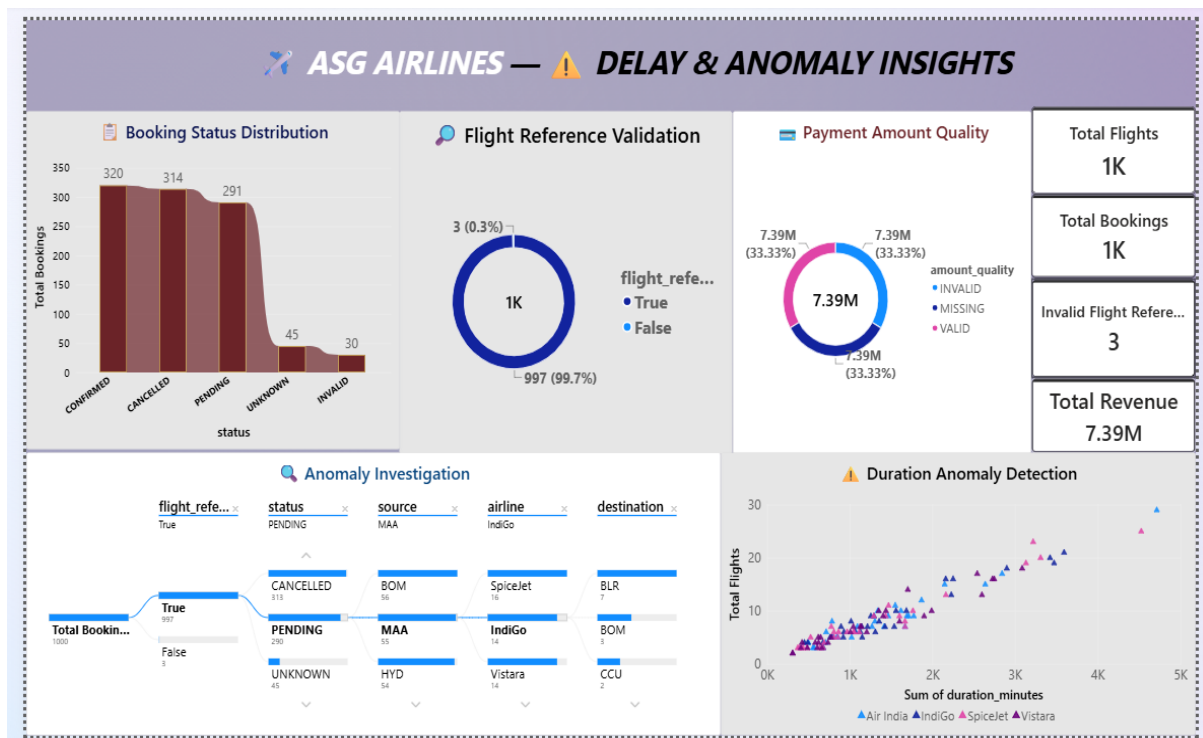


Figure 5 Delay & Anomaly Insights Dashboard 5

Description:

This dashboard is about finding unusual things and problems with the data in flight operations. It shows things like the status of bookings if flight references are correct if the payment amounts make sense if there are flight references and if there are any strange flight durations. The analysis breaks things down so you can look closely at each part, by airline, where the booking came from, where its going and what the booking status is.

14. PRIVACY AND PII PROTECTION

The bookings sheet contains personal fields – passport_number, seat_number, emergency_contact_name, and emergency_contact_phone – alongside passport-linked booking details. The passengers sheet contains each person's first and last name. None of this

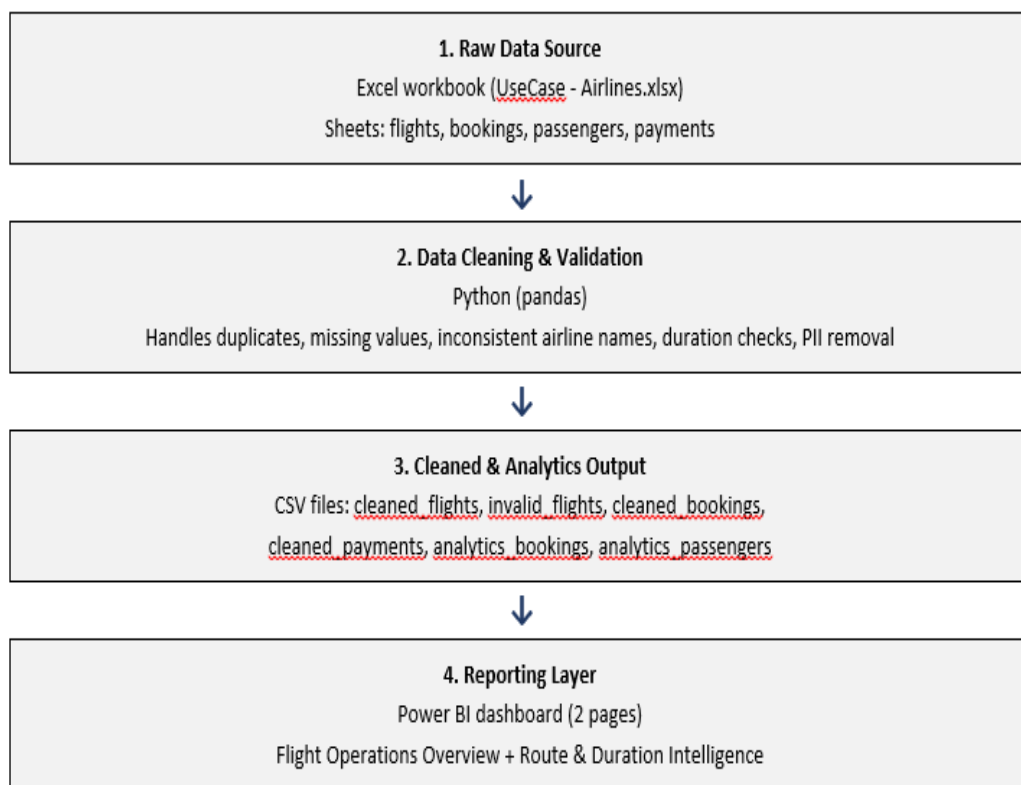
needs to be visible in a dashboard that is meant to show operational numbers like flight counts and revenue.

To handle this, I created separate “analytics-safe” versions of these two tables:

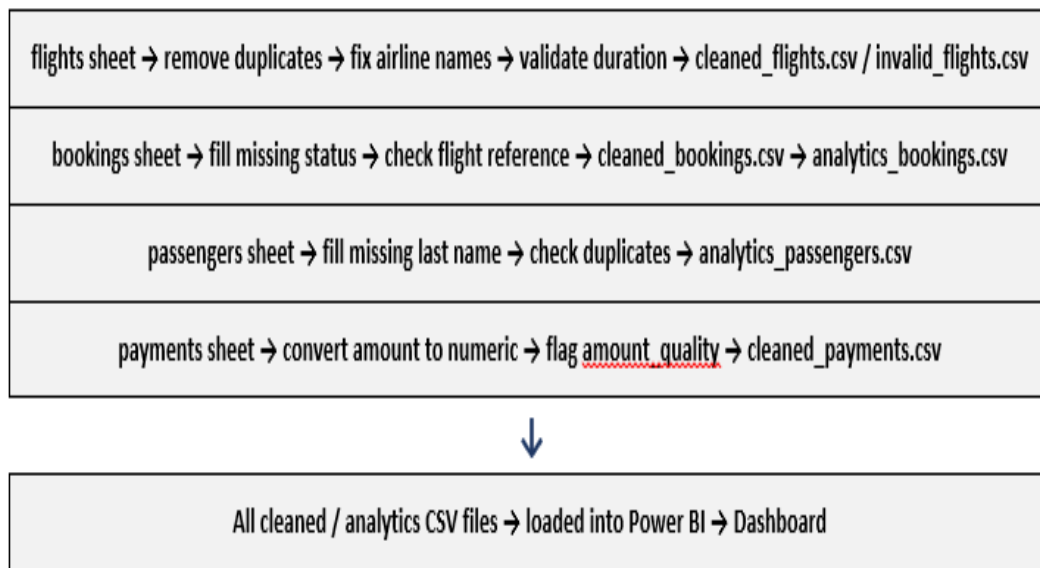
- analytics_bookings keeps only booking_id, passenger_id, flight_id, booking_date, status, and flight_reference_valid – the passport number and emergency contact fields are dropped entirely.
- analytics_passengers keeps only passenger_id, age, and gender – first name and last name are dropped.

These analytics-safe files are what actually get used for reporting and for the Power BI dashboard, while the fuller cleaned_bookings.csv (with the personal fields) is kept separately and is not something I fed into the reporting layer.

15. ARCHITECTURE DIAGRAM



16. DATAFLOW DIAGRAM



17. DATA MODEL



Relationships: one flight can have many bookings; one passenger can have many bookings; one booking can have one or more payments. BOOKINGS sits in the middle of the model since it is the table that ties everything else together.

18. ASSUMPTIONS

- I assumed that the first two characters of flight_id reliably identify the airline, since this pattern held consistently across the dataset, and used it to fix the inconsistent airline column.
- I assumed that the departure_time and arrival_time timestamps were generally reliable, and that only the specific records failing the duration/validation checks were actual errors, rather than assuming the whole dataset needed re-verification.
- I assumed that a duplicate flight_id (6F250) meant the record was unreliable rather than representing two genuinely different flights that happened to share an ID, so I excluded both occurrences rather than guessing which one was correct.
- I assumed that a missing booking status meant the status was simply not recorded, so I labelled it “UNKNOWN” rather than assuming a specific status like “PENDING”.
- I assumed that payment amounts that could not be converted to a number were data entry problems, not a different currency or format that needed separate handling.

19. CHALLENGES AND SOLUTIONS

CHALLENGES	SOLUTIONS
Airline column had UNKNOWN or inconsistent values	Derived the airline name from the flight_id prefix using a mapping dictionary instead of trusting the original column.
Duration field sometimes didn't match the timestamps	Recalculated duration from arrival_time minus departure_time and compared it with a small tolerance, instead of trusting the given duration directly.
One flight had an arrival time before its departure time	Flagged it through the duration validation check (negative duration) and moved it to the invalid_flights output rather than including it in the clean data.

Same flight_id (6F250) appeared twice	Detected duplicate flight IDs, removed both occurrences from the clean data, and re-checked which bookings were affected.
Payment amount column had non-numeric values	Converted the column with <code>pd.to_numeric(errors="coerce")</code> and added an <code>amount_quality</code> flag so revenue is only calculated from valid rows.
Personal data (passport numbers, contact details) mixed into the bookings table	Built separate analytics-safe tables with only the columns needed for reporting, and kept the personal fields out of the reporting layer.

20. SCALABILITY AND PERFORMANCE

This project worked with a fairly small dataset – about a thousand records per table – so everything was processed in memory using pandas without any performance issues. For a dataset of this size, this approach is simple and fast enough.

If ASG Airlines' actual data was much larger – for example, millions of flights and bookings – this in-memory, single-script approach would need to change. Some things I would consider for a bigger dataset: processing data in chunks instead of loading everything at once, moving the cleaning logic into a proper database or a scheduled pipeline instead of running a script manually, and storing the cleaned output in a database rather than CSV files so Power BI (or any other tool) can query it more efficiently.

21. CONCLUSION

Working through this project helped me understand what “messy” data actually looks like in practice – it's rarely one obvious problem, it's usually a mix of small inconsistencies (wrong airline labels, a mismatched duration, a duplicate ID, a bad payment amount) that all need to be caught before the numbers on a dashboard can be trusted.

The main things I took away from this project were how to validate data instead of just cleaning it blindly, how to think about overnight/cross-day timestamps correctly, and how to keep personal information separate from the data used for analytics and reporting. The final result is a set of cleaned CSV files and a two-page Power BI dashboard that gives a reasonably clear picture of ASG Airlines' flight operations.